

Relatório do Projeto

Simulador ARM: ARM-igos do Basseto

PCS 3732 – Laboratório de Processadores
12.08.2025



Gabriel Silva Vilas
NUSP: 13680150

Victor Pedreira dos Santos Pepe
NUSP: 13679565

Nicholas Sasaki Ogata
NUSP: 13680417

Conteúdo

1	Introdução	2
2	Visão Geral e Funcionalidades	3
2.1	Conjunto de Instruções Suportado	3
2.2	Estado da Máquina: Registradores e Memória	3
2.3	Funcionalidades e Interface (TUI)	4
2.4	Arquitetura do Software	4
3	Módulos do Simulador	6
3.1	O Módulo de Memória	6
3.1.1	Representação e Estrutura	6
3.1.2	Implementação (memoria.cpp)	6
3.1.3	Carregamento do Programa	7
3.1.4	Little-Endian e Acesso a Palavras	7
3.2	O Módulo Decodificador	7
3.2.1	Estrutura e Fluxo de Decodificação	8
3.2.2	Decodificação de Instruções de Dados	8
3.2.3	Decodificação de Acesso à Memória	9
3.2.4	Decodificação de Saltos	9
3.3	O Módulo Executor	10
3.3.1	Estrutura e Fluxo de Execução	10
3.3.2	Tratamento de Operandos e a Emulação do Pipeline . .	10
3.3.3	Implementação das Instruções de ULA	10
3.3.4	Implementação de Acesso à Memória e Saltos	11
3.4	O Módulo CPU	11
3.4.1	Representação do Estado Interno	12
3.4.2	O Ciclo de Execução: Fetch-Decode-Execute	12
3.4.3	Execução Condicional e Manipulação de Flags	13
4	Como Usar o Simulador	14
4.1	Passo 1: Escrevendo o Código Assembly	14
4.2	Passo 2: Compilando com o Makefile	14
4.3	Passo 3: Executando a Simulação	15
4.4	Exemplo de uso	15

1 Introdução

Este relatório apresenta o nosso projeto final para a disciplina de Laboratório de Processadores: um simulador para a arquitetura ARMv7, desenvolvido em C++. O nome que demos ao projeto foi "*ARM-igos do Basseto*", um trocadilho com o nome do nosso querido professor, Bruno Basseto, o grande responsável por transmitir o denso e complexo conteúdo dessa disciplina de forma leve e didática ao longo dos últimos 4 meses.

A ideia do projeto foi simples: colocar em prática tudo o que aprendemos na matéria. Em vez de apenas escrever código Assembly, nós queríamos entender o que acontece do outro lado. Como o processador realmente lê aqueles bytes e os transforma em ações? Para responder a essa pergunta, decidimos construir nosso próprio processador virtual.

A importância de criar o simulador foi a mudança de perspectiva. Tivemos que lidar com os desafios que um processador real enfrenta: decodificar instruções binárias, gerenciar um banco de registradores, simular a memória, controlar os flags de condição e implementar a lógica de cada instrução, desde um simples 'ADD' até um 'LDR' com endereçamento. Foi a melhor forma de consolidar nosso conhecimento sobre o ciclo de *Fetch-Decode-Execute* e entender como o software de baixo nível e o hardware realmente conversam. Todo o desenvolvimento do projeto está disponível nesse [link do GitHub](#).

2 Visão Geral e Funcionalidades

O projeto "*ARM-igos do Baseto*" consiste em um programa de linha de comando, desenvolvido em C++, que simula o funcionamento de um processador ARMv7 de 32 bits em um modelo de ciclo único. O objetivo é carregar e executar um programa em código de máquina, compilado para a arquitetura ARM, em um ambiente "bare-metal" simulado, permitindo a visualização e depuração do seu estado interno a cada passo.

2.1 Conjunto de Instruções Suportado

Para manter o escopo do projeto viável, implementamos um subconjunto básico de instruções do ARMv7, focando nas operações mais essenciais:

- **Instruções de ULA (Processamento de Dados):** Foram implementadas as principais operações aritméticas e lógicas, como ADD, SUB, AND, ORR, EOR, MOV e CMP. O simulador suporta a atualização dos flags de condição através do sufixo S (ex: ADDS).
- **Barrel Shifter:** A funcionalidade do Barrel Shifter foi implementada para o segundo operando das instruções de ULA, permitindo operações de deslocamento (LSL, LSR, ASR) e rotação (ROR) sobre operandos de registradores.
- **Instruções de Acesso à Memória:** O simulador suporta as instruções LDR e STR para a transferência de palavras de 32 bits entre a memória e os registradores. Foram implementados os modos de endereçamento mais comuns, como o indireto por registrador e o com deslocamento imediato.
- **Instruções de Controle de Fluxo:** Foram implementadas as instruções de salto B (Branch) e BL (Branch with Link), incluindo o suporte para todas as variantes de saltos condicionais (ex: BEQ, BNE, BGT).

2.2 Estado da Máquina: Registradores e Memória

O simulador mantém e gerencia o estado completo de um processador simplificado:

- **Banco de Registradores:** É simulado um banco com os 16 registradores de propósito geral de 32 bits (R0 a R15), respeitando as funções especiais do R13 (SP), R14 (LR) e R15 (PC).
- **Memória RAM:** A memória principal é representada por um vetor de bytes de tamanho configurável (64KB em nossos testes), onde o código e os dados do programa a ser simulado são carregados.
- **Flags de Condição (CPSR):** O simulador gerencia o estado dos quatro flags de condição principais – N (Negative), Z (Zero), C (Carry) e V (Overflow) –, atualizando-os conforme as instruções são executadas.

2.3 Funcionalidades e Interface (TUI)

Para tornar a interação e a depuração do simulador uma experiência mais rica e didática, foi desenvolvida uma Interface de Usuário baseada em Texto (TUI) utilizando a biblioteca *ncurses*. A TUI apresenta um "dashboard" em tempo real com as seguintes funcionalidades:

- **Execução Passo a Passo:** O usuário pode executar o programa instrução por instrução, pressionando uma tecla para avançar para o próximo ciclo.
- **Visualização de Estado:** A tela é dividida em painéis que exibem continuamente o conteúdo dos registradores, o estado dos flags do CPSR e o código em memória ao redor do Program Counter.
- **Navegação na Memória:** O usuário pode rolar a visualização do painel de memória para cima e para baixo usando as teclas de seta, permitindo inspecionar qualquer região da RAM simulada.

2.4 Arquitetura do Software

O simulador foi arquitetado em C++ de forma modular, com uma clara separação de responsabilidades para facilitar o desenvolvimento e a depuração:

- **Módulo CPU ('cpu.cpp'):** A classe central que guarda o estado da máquina (registradores, CPSR) e orquestra o ciclo de *Fetch-Decode-Execute*.

- **Módulo de Memória ('memoria.cpp'):** Classe responsável por gerenciar a RAM simulada, incluindo o carregamento do programa e o acesso seguro aos dados.
- **Módulo Decodificador ('decodificador.cpp'):** Isola toda a complexa lógica de manipulação de bits necessária para traduzir uma instrução binária de 32 bits em uma estrutura de dados compreensível.
- **Módulo Executor ('executor.cpp'):** Contém a implementação da semântica de cada instrução. Ele recebe uma instrução decodificada e modifica o estado da CPU de acordo.
- **Módulo de Visualização ('tuiviewer.cpp'):** Isola toda a lógica de interface com a biblioteca *ncurses*, sendo responsável por desenhar o "dashboard" na tela.

A arquitetura e como esses módulos se relacionam está melhor representada na figura 1 (não colocamos o tuiviewer, pois ele não faz parte da estrutura ARM em si).

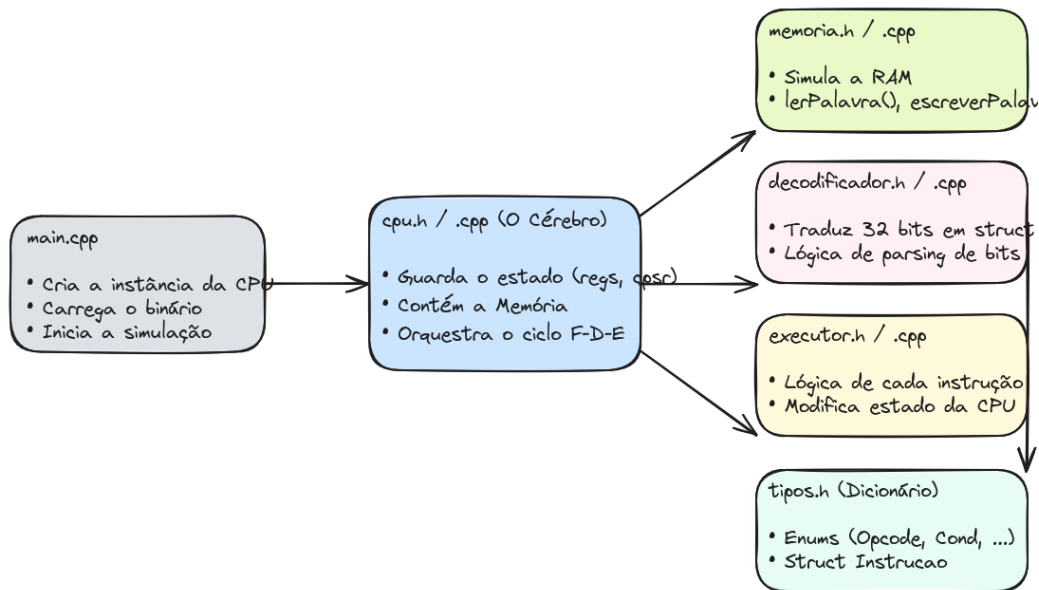


Figura 1: Arquitetura dos módulos do simulador ARM

3 Módulos do Simulador

A seguir, serão apresentados cada um dos módulos desenvolvidos para o simulador, sendo cada um deles responsável por uma tarefa específica (ou então por agregar os outros módulos). São eles: **Memória**, **Decodificador**, **Executor e CPU**. Há também um módulo de visualização, o **TUIViewer**. Entretanto, ele é apenas um adicional para a interface do simulador (não interfere na arquitetura nem no desenvolvimento do simulador em si), então não entraremos em detalhes sobre a sua implementação.

3.1 O Módulo de Memória

A simulação de um processador seria inútil sem uma memória para armazenar o código e os dados. O Módulo de Memória é o componente do nosso simulador responsável por replicar o comportamento de uma memória RAM, fornecendo uma interface controlada para que a CPU possa realizar operações de leitura e escrita.

3.1.1 Representação e Estrutura

A memória foi implementada em C++ utilizando um `std::vector<uint8_t>`, ou seja, um vetor de bytes sem sinal. Essa escolha oferece duas vantagens principais:

- **Endereçamento por Byte:** Permite que cada byte individual na memória simulada tenha um endereço único, que é o comportamento padrão de sistemas de computação reais.
- **Alocação Dinâmica:** O tamanho da RAM simulada pode ser facilmente definido no momento da criação do objeto, como fizemos com 64KB em nossos testes.

3.1.2 Implementação (`memoria.cpp`)

A implementação em `memoria.cpp` contém a lógica para carregar programas e, mais importante, para manipular os dados em um formato que o processador ARM entende.

3.1.3 Carregamento do Programa

O método `carregarDeArquivo` é responsável por ler um arquivo binário cru (gerado pelo `objcopy`) e copiá-lo, byte a byte, para o vetor `ram` a partir de um endereço inicial especificado (em nosso caso, `0x8000`).

3.1.4 Little-Endian e Acesso a Palavras

A arquitetura ARM opera com a ordem de bytes *little-endian*. Nossos métodos `lerPalavra` e `escreverPalavra` implementam essa lógica.

Ao ler uma palavra do endereço `A`, os bytes são combinados da seguinte forma:

```
1 uint32_t byte0 = ram[A];
2 uint32_t byte1 = ram[A + 1];
3 uint32_t byte2 = ram[A + 2];
4 uint32_t byte3 = ram[A + 3];
5
6 uint32_t palavra = byte0 | (byte1 << 8) | (byte2 << 16) | (
    byte3 << 24);
```

Ao escrever uma palavra, o processo é invertido, quebrando o valor de 32 bits em 4 bytes:

```
1 ram[A] = valor & 0xFF;
2 ram[A + 1] = (valor >> 8) & 0xFF;
3 ram[A + 2] = (valor >> 16) & 0xFF;
4 ram[A + 3] = (valor >> 24) & 0xFF;
```

Além disso, ambos os métodos realizam verificações de segurança para garantir que os acessos à memória estejam dentro dos limites do vetor `ram` e que os endereços para leitura/escrita de palavras estejam alinhados em 4 bytes, lançando uma exceção caso contrário, o que simula o comportamento de *Data Abort* de um processador real.

3.2 O Módulo Decodificador

O Módulo Decodificador é o componente cuja única responsabilidade é receber uma instrução binária de 32 bits e convertê-la em uma estrutura de dados (`Struct Instrucao`) rica em informações semânticas, pronta para ser utilizada pelo Módulo Executor.

Toda a lógica de manipulação de bits, como a aplicação de máscaras (`&`) e deslocamentos (`>>`), foi isolada neste módulo. Isso torna o resto do código

do simulador, especialmente o Executor, muito mais limpo e legível, pois ele pode operar com base em conceitos como "opcode" e "registradores" em vez de bits individuais.

3.2.1 Estrutura e Fluxo de Decodificação

A implementação do decodificador, contida em `decodificador.cpp`, foi projetada de forma modular. A função principal, `decodificar()` analisa os bits 27 a 25 da instrução para determinar a qual categoria geral ela pertence:

- **Processamento de Dados:** Instruções de ULA como ADD, SUB, MOV, etc.
- **Acesso à Memória:** Instruções LDR e STR.
- **Salto:** Instruções de controle de fluxo B e BL.

Com base nessa identificação inicial, a função principal delega o trabalho a uma função auxiliar especializada, como `decodificarProcessamentoDeDados()`, que conhece os detalhes do formato daquela categoria de instrução.

3.2.2 Decodificação de Instruções de Dados

Esta é a categoria de instrução mais complexa devido à flexibilidade do segundo operando do ARM. A função `decodificarProcessamentoDeDados` extrai os seguintes campos:

- **Condição (bits 31-28):** Os quatro bits que definem a condição de execução (ex: EQ, NE).
- **Opcode (bits 24-21):** Os quatro bits que definem a operação a ser realizada (ex: 0b0100 para ADD).
- **Flag S (bit 20):** Um único bit que indica se a instrução deve ou não atualizar os flags do CPSR.
- **Registradores (Rn e Rd):** Os números dos registradores de operando e destino.
- **Segundo Operando:** O decodificador analisa o **bit I (bit 25)** para determinar se o segundo operando é um valor imediato ou um registrador.

- Se for **imediato**, ele extrai o valor base de 8 bits e o fator de rotação de 4 bits, e já aplica a rotação para gerar o valor final de 32 bits.
- Se for um **registrador**, ele extrai o número do registrador (R_m), o tipo de shift e a quantidade de deslocamento a ser aplicada pelo Barrel Shifter.

3.2.3 Decodificação de Acesso à Memória

A função `decodificarAcessoMemoria` extrai os campos específicos das instruções LDR e STR, que controlam o modo de endereçamento:

- **Bit L (20):** Define se a operação é um Load (LDR, L=1) ou um Store (STR, L=0).
- **Bit P (24):** Define o modo de indexação: Pré-Indexado (P=1) ou Pós-Indexado (P=0).
- **Bit U (23):** Define se o offset deve ser somado (U=1) ou subtraído (U=0) do registrador base.
- **Bit W (21):** Define se o *write-back* está habilitado, ou seja, se o registrador base deve ser atualizado com o endereço final.

3.2.4 Decodificação de Saltos

Finalmente, a função `decodificarSalto` lida com as instruções de controle de fluxo B e BL.

- **Bit L (24):** Link bit, diferencia uma instrução B (L=0) de uma BL (L=1).
- **Offset de 24 bits:** O decodificador extrai os 24 bits inferiores da instrução, que representam o deslocamento do salto. A implementação realiza a extensão de sinal para tratar corretamente os saltos para trás (offsets negativos) e multiplica o resultado por 4, pois o offset é armazenado como uma contagem de palavras, não de bytes.

3.3 O Módulo Executor

Sua responsabilidade é receber a instrução já decodificada e estruturada, e efetivamente executar a operação, modificando o estado da CPU (registradores, flags e memória) de acordo com a semântica da arquitetura ARM.

Toda a lógica de execução está contida no arquivo `executor.cpp`, que opera diretamente sobre o objeto `CPU` (passado por referência).

3.3.1 Estrutura e Fluxo de Execução

A principal função do módulo, `executarInstrucao` utiliza uma estrutura `switch` para avaliar o `opcode` da instrução recebida e, com base nisso, chama uma função auxiliar específica e dedicada para aquela operação (ex: `executar_add`, `executar_ldr`, etc.).

3.3.2 Tratamento de Operandos e a Emulação do Pipeline

Um dos desafios centrais do Executor é lidar com a flexibilidade dos operandos do ARM e emular corretamente o comportamento do Program Counter (PC).

- **Cálculo do Operando 2:** A função `calcularOperando2` é responsável por determinar o valor do segundo operando. Ela verifica se a instrução usa um valor imediato (que é retornado diretamente) ou um registrador. No caso de um registrador, ela aplica a operação do Barrel Shifter (LSL, LSR, ASR, ROR) antes de retornar o valor final.
- **Emulação do PC ('getValorOperando'):** Como nosso simulador é de ciclo único e incrementa o PC no início do ciclo de fetch, o valor de `r[15]` está sempre em `endereço_atual + 4`. No entanto, a arquitetura ARM se comporta como se o PC estivesse 8 bytes à frente (`endereço_atual + 8`). A função `getValorOperando` resolve essa discrepância: sempre que um operando requisitado é o PC (registrador 15), ela retorna o valor de `cpu.r[15] + 4`, garantindo que todos os cálculos relativos ao PC sejam corretos.

3.3.3 Implementação das Instruções de ULA

As funções de ULA (ex: `executar_add`) seguem um padrão claro:

1. Obter os valores dos operandos de origem (usando `getValorOperando` e `calcularOperando2`).
2. Realizar a operação matemática ou lógica. Para as operações aritméticas, utilizamos um tipo de 64 bits (`uint64_t`) para o resultado intermediário, o que facilita enormemente a detecção dos flags de Carry e Overflow.
3. Armazenar o resultado de 32 bits no registrador de destino (`'rd'`).
4. Se o *S-flag* da instrução estiver ativo, os flags N, Z, C e V do `cpsr` da CPU são atualizados. A lógica para o flag de Overflow (V), por exemplo, foi implementada usando a conhecida operação de bits:

```

1 bool overflow = (~(op1 ^ op2) & (op1 ^ cpu.r[instr.rd]))
  >> 31;
2 cpu.setFlagV(overflow);
3

```

A instrução `CMP` foi implementada de forma similar a `SUB`, com a diferença de que ela apenas atualiza os flags, sem armazenar o resultado da subtração.

3.3.4 Implementação de Acesso à Memória e Saltos

- **LDR e STR:** As funções `executar_ldr` e `executar_str` calculam o endereço final de memória (`base + offset`) e então chamam os métodos `lerPalavra` ou `escreverPalavra` do Módulo de Memória. Elas também implementam a lógica de *write-back* quando o bit `W` da instrução está setado.
- **B e BL:** As instruções de salto modificam diretamente o PC (`cpu.r[15]`). A função `executar_b` implementa a fórmula de salto relativo, somando o offset decodificado ao valor do PC emulado (`r[15] + 4`). A função `executar_bl` primeiro salva o endereço de retorno (o valor atual de `r[15]`) no Link Register (`r[14]`) antes de realizar o mesmo cálculo de salto.

3.4 O Módulo CPU

A classe `CPU`, definida em `cpu.cpp`, é o componente central do nosso simulador. Ela não se aprofunda nos detalhes de *como* decodificar ou executar

uma instrução, mas ordena todos os demais componentes: ele **guarda todo o estado da máquina** e **orquestra o ciclo de vida** de uma instrução, delegando as tarefas específicas para os módulos Decodificador e Executor.

3.4.1 Representação do Estado Interno

Para simular um processador, a classe `CPU` encapsula todos os elementos que compõem o estado arquitetural de um núcleo ARM simples.

- **Banco de Registradores (`r`):** Um `std::array<uint32_t, 16>` que representa os registradores de propósito geral de R0 a R15. O acesso a registradores especiais como o PC e o SP é feito através de seus índices (15 e 13, respectivamente).
- **Registrador de Status (`cpsr`):** Um único inteiro de 32 bits (`uint32_t`) que armazena os quatro flags de condição (N, Z, C, V) em seus quatro bits mais significativos. A manipulação desses flags é feita através de métodos auxiliares.
- **Memória (`memoria`):** Uma instância da nossa classe `Memoria`, contida dentro da `CPU`. A `CPU` interage com este objeto para buscar instruções e para as operações de Load/Store.

3.4.2 O Ciclo de Execução: Fetch-Decode-Execute

O método mais importante da classe é o `bool CPU::passo()` que implementa uma única iteração do ciclo. A cada chamada, ele executa os três estágios:

1. **Fetch:** A `CPU` usa o endereço armazenado em seu Program Counter (`r[15]`) para ler uma palavra de 32 bits do seu objeto `Memoria`. Imediatamente após a busca, o PC é incrementado em 4 para já apontar para a próxima instrução sequencial.
2. **Decode:** A palavra binária de 32 bits, que representa a instrução, é enviada para a função externa `decodificar()`. Esta função, pertencente ao Módulo Decodificador, retorna uma `struct Instrucao` com todos os campos já "traduzidos" e prontos para uso.
3. **Execute:** Antes de executar, a `CPU` primeiro chama seu método privado `checarCondicao()`, passando o campo de condição da instrução

decodificada. Se a condição for satisfeita, a CPU então invoca a função externa `executarInstrucao()`, passando a si mesma por referência (`*this`) e a instrução decodificada. Isso permite que o Módulo Executor realize a operação e modifique o estado da CPU. Se a condição não for satisfeita, nenhuma ação é tomada e a instrução é efetivamente ignorada.

3.4.3 Execução Condicional e Manipulação de Flags

A arquitetura ARM permite que quase toda instrução seja condicional. Essa lógica é implementada no método `bool CPU::checarCondicao(Condicao cond)`.

Ele consiste em uma estrutura `switch` que avalia o `enum Condicao` da instrução. Para isso, ele primeiro obtém o estado atual dos flags através de métodos auxiliares como `getFlagN()`, que isolam a lógica de manipulação de bits. Com base nos valores dos flags, o método retorna `true` ou `false`, determinando se o Executor será chamado. Da mesma forma, os métodos `setFlagN/Z/C/V` permitem que o Executor modifique o `cpsr` de forma segura e abstrata após operações que alteram os flags.

4 Como Usar o Simulador

Esta seção descreve o fluxo de trabalho para escrever, montar e executar um programa em Assembly ARM no nosso simulador. O processo é dividido em três etapas: a escrita do código-fonte, a montagem e dumpagem via `Makefile`, e a execução do simulador com o binário resultante.

4.1 Passo 1: Escrevendo o Código Assembly

O primeiro passo consiste em criar um arquivo-fonte com a extensão `.s` (por exemplo, `program.s`). Este arquivo deve conter as instruções em Assembly ARMv7 que se deseja simular.

É fundamental que o código a ser executado primeiro, marcado com o rótulo global `_start`, esteja fisicamente no início do arquivo. Isso é uma necessidade imposta pela simplicidade do nosso simulador, que carrega um binário cru e começa a execução a partir do primeiro byte, sem analisar cabeçalhos ou pontos de entrada formais do formato ELF. O código pode utilizar as seções `.data` e `.bss` para declarar variáveis, que serão corretamente posicionadas na memória pelo linker script.

4.2 Passo 2: Compilando com o Makefile

Com o código-fonte pronto, o processo de compilação é totalmente automatizado pelo `Makefile` fornecido com o projeto. Ao ser invocado, ele executa a sequência de tarefas com o `arm-none-eabi`:

1. **Montagem:** O montador (`as`) converte o código-fonte `.s` em um arquivo objeto `.o`, contendo o código de máquina ainda em um formato relocável.
2. **Linkagem:** O linker (`ld`) utiliza o nosso `linker.ld` para juntar os arquivos objeto, resolver os endereços dos símbolos e gerar um arquivo executável final no formato ELF.
3. **Conversão:** Por fim, o utilitário `objcopy` é chamado com a flag `-O binary`. Ele extrai apenas o código de máquina e os dados inicializados do arquivo ELF, descartando todos os metadados, para criar uma imagem binária crua (`.img`), que é o formato que nosso simulador lê.

Para realizar todas essas etapas, o usuário precisa apenas executar um único comando no terminal, na pasta do projeto: `make`

Para limpar todos os arquivos gerados pela compilação, o comando é: `make clean`

4.3 Passo 3: Executando a Simulação

Após a compilação bem-sucedida, a imagem binária estará pronta para ser simulada. Para executar o simulador, o usuário deve passar o caminho para o arquivo `.img` gerado como um argumento de linha de comando.

Ao ser executado, o simulador irá carregar o binário para sua memória interna e apresentar a Interface de Usuário em Texto (TUI). A partir daí, o usuário pode controlar a execução de forma interativa, avançando instrução por instrução, observando as alterações nos painéis de registradores e flags, e navegando pela memória para verificar os resultados das operações.

4.4 Exemplo de uso

Considere, inicialmente, que estamos dentro do repositório, em sua base. A árvore de arquivos esá como na figura 2.

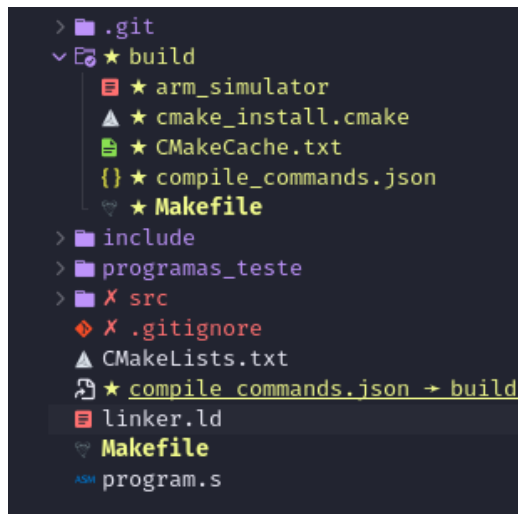


Figura 2: Árvore de arquivos na base do repositório

Como exemplo, vamos utilizar o programa `program.s` que apenas carrega

um valor da memória para R0 e realiza um simples branch para o fim do programa (instrução B end).

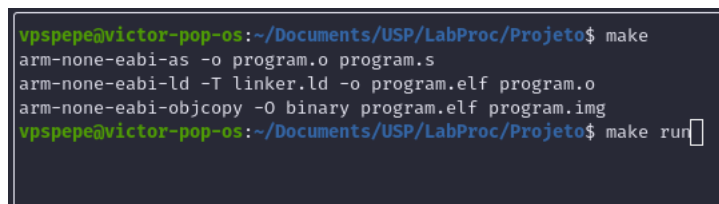
Listing 1: Código Assembly do program.s.

```

1 .section .data
2 dado:
3 .word 0x4, 0x6, 0x8
4
5 .section .text
6 .global _start
7
8 _start:
9     MOV R0, #1
10    ldr r4, =dado
11    ldr R0, [r4]
12    B in
13    mov r1, #2
14    mov r3, #4
15 in:
16    MOV R2, #3
17    B end
18 end:
19    MOV R0, #3
20    b .

```

Basta agora digitar `make`, no terminal, para realizar a montagem e a dumpagem. Em seguida, executar `make run` para executar o simulador, já com o programa carregado.



```

vpspepe@victor-pop-os:~/Documents/USP/LabProc/Projeto$ make
arm-none-eabi-as -o program.o program.s
arm-none-eabi-ld -T linker.ld -o program.elf program.o
arm-none-eabi-objcopy -O binary program.elf program.img
vpspepe@victor-pop-os:~/Documents/USP/LabProc/Projeto$ make run

```

Figura 3: Montagem, Dumpagem e Execução com Make

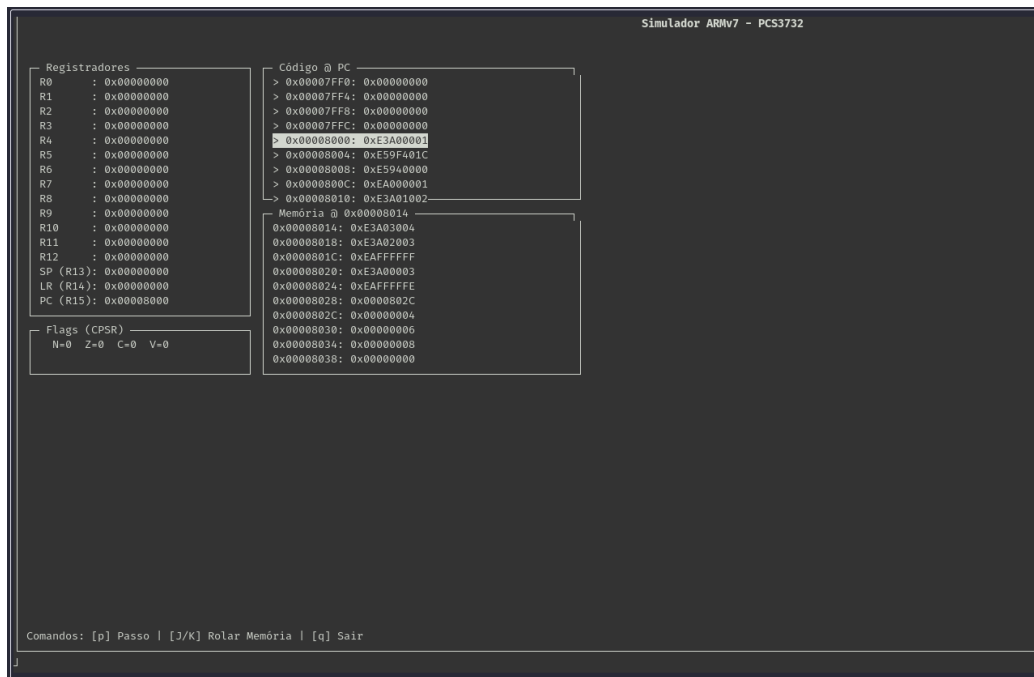


Figura 4: Interface Gráfica com execução do simulador

Note que, na interface gerada no terminal, é possível visualizar o valor de todos os registradores - incluindo o CPSR e suas flags -, o código sendo executado no momento e o conteúdo da memória (que pode ser varrido para cima e para baixo com as teclas J e K). Com a tecla P, é possível avançar na execução, e com a tecla Q, encerra-se o simulador.